Live autotests — setup and run

This document describes how to run the OmegaClaw autotest suite against a real LLM provider (Anthropic / ASICloud / ASIOne / OpenAI / OpenRouter / Ollama-local), as opposed to the deterministic LLM mock used by the variants under `Autotests/mock/`, `Autotests/mock_telegram/`, and `Autotests/mock_memory/`. The live and mock variants share the same set of assertions; the difference is the source of the agent's response.

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.
- A valid API key for the chosen LLM provider.

2. Container deployment

Run on the QA host before starting pytest. Replace placeholders in angle brackets with concrete values. `<PROVIDER_KEY_ENV>` and `<PROVIDER_NAME>` must agree (Anthropic + `ANTHROPIC_API_KEY`, ASICloud + `ASI_API_KEY`, OpenAI + `OPENAI_API_KEY`).

```
docker run -d -it \
  --name <CONTAINER_NAME> \
  --user 65534:65534 \
  --init \
  --network bridge \
  --security-opt no-new-privileges:true \
  --volume omegaclaw-memory:/PeTTa/repos/OmegaClaw-Core/memory/ \
  --tmpfs /tmp:size=64m,mode=1777,exec \
  --tmpfs /run:size=16m,mode=755 \
  --tmpfs /var/tmp:size=64m,mode=1777,exec \
  -e <PROVIDER_KEY_ENV>="<KEY>" \
  -e OMEGACLAW_AUTH_SECRET=0000 \
  singularitynet/omegaclaw:<IMAGE_SHA> \
  IRC_channel="<CHANNEL>" \
  provider="<PROVIDER_NAME>" \
  embeddingprovider="Local"
```

Parameters

Parameter	Required	Description
`--name <CONTAINER_NAME>`	Yes	Container name. Must equal `OMEGACLAW_CONTAINER` on the host shell.
`--user 65534:65534`	No	Runs the agent as `nobody:nogroup`. Recommended; matches the image default.
`--init`	Yes	Uses tini as PID 1 to reap zombie processes spawned by `(run …)` and `(create-script …)`.

`--network bridge`	Yes	Outbound network for IRC, LLM API, search, embeddings.
`--volume omegaclaw-memory:<…/memory/>`	Yes	Persists ChromaDB and `history.metta` across container restarts. Tests run fine without it within one session.
`--tmpfs /tmp:…,exec`	Yes	`exec` is mandatory: several tests create shell scripts in `/tmp` and execute them. Default Docker tmpfs is `noexec`.
`--tmpfs /var/tmp:…,exec`	Yes	Same as `/tmp`; some tests use `/var/tmp`.
`--tmpfs /run:…`	No	System runtime files; no scripts run from here.
`-e <PROVIDER_KEY_ENV>`	Yes	API key for the chosen LLM provider (`ANTHROPIC_API_KEY` / `ASI_API_KEY` / `OPENAI_API_KEY`).
`-e OMEGACLAW_AUTH_SECRET=0000`	Yes	IRC authorization secret. The first nick that sends `auth <secret>` binds; later nicks are ignored. `helpers.py` sends `auth 0000` hard-coded, so the value must stay `0000`.
`singularitynet/omegaclaw:<IMAGE_SHA>`	Yes	Image pinned by full SHA. Tests are written against a specific build of skill semantics.
`IRC_channel="<CHANNEL>"`	Yes	Channel the agent joins on `irc.quakenet.org`. Must equal `OMEGACLAW_IRC_CHANNEL` on the host shell.
`provider="<PROVIDER_NAME>"`	Yes	LLM provider: `Anthropic` / `ASICloud` / `OpenAI`. Must match the API key supplied above.
`embeddingprovider="Local"`	Yes	Local sentence-transformers in-container. OpenAI is also supported but consumes credits and adds latency.

3. Environment for pytest

Export these on the host shell before running pytest:

```
export OMEGACLAW_CONTAINER="<CONTAINER_NAME>"
export OMEGACLAW_IRC_CHANNEL="<CHANNEL>"
export OMEGACLAW_GIT_TOKEN="<GITHUB_PAT>"
```

Variable	Required	Description
`OMEGACLAW_CONTAINER`	Yes	Name passed to `docker exec`. Must equal `--name` above. Default in `helpers.py` is `omegaclaw`.
`OMEGACLAW_IRC_CHANNEL`	Yes	Channel for `PRIVMSG`. Must equal `IRC_channel` above. Default in `helpers.py` is `#metaclaw777`.
`OMEGACLAW_GIT_TOKEN`	No	GitHub PAT used by `test_git_push_to_remote`. The test is skipped if this variable is unset.

`OMEGACLAW_GIT_REMOTE`	No	Override the test remote. Default `https://github.com/OmegaSing/Test-Repopo`.

4. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v test_*.py
```

5. General notes

- All files required for checks are created at the beginning of the test before accessing OmegaClaw. Upon completion of the tests (both positive and negative), all test files are deleted.
- At the beginning of each test it is verified that there are no leftover test files in the target directories.
- The tests verify functionality of the agent's skills. Each skill has several core functions that are covered by tests, either individually or in combination.
- When launching the container, use Authorization Key `0000`. To use a different value, edit the line `sock.sendall(f"PRIVMSG {CHANNEL} :auth 0000\r\n".encode())` in `helpers.py` before starting the tests.
- Tests will be expanded as we go deeper into the project.

6. Grading: 0 / 1 / 2

Tests where the agent's success may legitimately depend on prompt phrasing (complex multi-step tasks, network-dependent searches, git workflows) report a grade in addition to pass/fail. The grade is set via `Checker.set_grade()` in `helpers.py`:

Grade	Meaning
1 — FIRST TRY	Agent solved the task on the original prompt within the first timeout window.
2 — AFTER CLARIFY	First attempt timed out; the harness sent a clarification prompt and the agent succeeded on the retry.
0 — FAIL	Agent did not produce the expected result even after the clarification prompt.

Mechanics live in `try_with_clarification()` in `helpers.py`. If a graded step succeeds but a later step in the same test fails, the grade is collapsed to 0 so it always matches the test outcome. Tests without a graded step are reported as a plain pass/fail and never print a grade.

The graded tests: `test_create_script`, `test_convert_format`, `test_run_error_script`, `test_run_create_dirs`, `test_search_weather`, `test_git_pull_public`, `test_git_local_commit`, `test_git_push_to_remote`, `test_memory_episode`, `test_complex_weather_flow`.

7. Single-skill tests (one skill in isolation)

These tests exercise one OmegaClaw skill at a time. `(send …)` is treated as the reply transport, not as a second skill under test.

Creating files

1. test_create_file.py

Creates `/tmp/testcat/hello.txt` containing exactly `Hello`.

- Skill: write-file.
- Checks: directory exists, file exists, permissions start with `-rw`, mtime ≥ test start, content is `Hello` (with or without trailing newline).
- Grade: not graded.

2. test_create_empty_file.py

Creates an empty `/tmp/test_empty/hello.txt`.

- Skill: write-file.
- Checks: file is created, `cat` returns an empty string.
- Grade: not graded.

3. test_create_script.py

Creates `/tmp/test_script/date.sh` — a shell script that prints the current date.

- Skill: write-file (with executable bit; agent may use shell chmod).
- Checks: file exists, is executable (`x` in permissions), the harness runs it via `sh date.sh` and verifies the output contains the current year (UTC or local).
- **GRADED.** First attempt 120 s, post-clarification 180 s.

Editing files

4. test_edit_add_timestamp.py

Appends the current timestamp as a new line to a pre-created `note.txt` containing "This is a test note."

- Skill: append-file (or write-file rewrite).
- Preparation: harness creates the file with one line.
- Checks: mtime changed, last line of the file contains ≥ 4 digits.
- Grade: not graded.

5. test_edit_delete_line.py

Deletes the second line from a pre-created 3-line file (First line / Second line / Third line).

- Skill: write-file (rewrite).
- Checks: mtime changed, exactly 2 lines remain, lines 1 and 3 intact, line 2 is gone.
- Grade: not graded.

6. test_edit_append_line.py

Appends a 4th line `Delta` to a pre-created 3-line file (Alpha / Bravo / Charlie).

- Skill: append-file.
- Checks: mtime changed, 4 lines total, first 3 lines unchanged, 4th line equals `Delta`.
- Grade: not graded.

7. test_convert_format.py

Converts `document.md` → `document.txt`, preserving textual content.

- Skill: shell (e.g. `cp src dst`) or write-file (read source then rewrite as `.txt`).
- Preparation: harness creates the `.md` file with title, paragraph, and a bullet list.
- Checks: `.txt` file exists, contains the keywords `My Title`, `Some paragraph text`, `item one`, `item two`.
- **GRADED.** First attempt 120 s, post-clarification 180 s.

Running shell scripts

8. test_run_error_script.py

Runs a syntactically broken script and captures stdout + stderr to a file.

- Skill: shell (with redirection `2>&1`).
- Preparation: harness pre-creates `broken.sh` with an unclosed `if [`.
- Checks: `output.txt` exists, contains the literal `start` (stdout) AND at least one of `error` / `syntax` / `unexpected` / `missing` / `not found` (stderr), container is still alive.
- **GRADED.** First-attempt timeout 60 s, post-clarification 120 s.

9. test_run_repeated.py

Runs `dateupdate.sh` exactly 10 times in a row.

- Skill: shell.
- Preparation: harness pre-creates a script that appends date to `update.txt`.
- Checks: `update.txt` exists with mtime ≥ start, has ≥ 10 lines, every line contains date-like digits.
- Grade: not graded.

Internet search

10. test_search_basic.py

Sends prompt "What is SingularityNet? Search the web."

- Skill: search or tavily-search.
- Checks: agent invoked search/tavily-search with `singularity` in the argument; the reply contains at least one of `singularitynet`, `agi`, `blockchain`, `decentralized`, `marketplace`, `goertzel`.
- Grade: not graded.

11. test_search_weather.py

Cross-checks Valencia's temperature against the open-meteo.com API.

- Skill: search or tavily-search.
- Preparation: harness fetches reference temperature from open-meteo.
- Checks: agent invoked search/tavily-search with `valencia` in the argument; the reply contains a Celsius number within ±10 °C of the reference.
- **GRADED** (search-call stage). First attempt 120 s, post-clarification 180 s. The follow-up `(send ...)` stage uses a separate 240 s window (not graded itself; collapses overall grade to 0 on timeout).

12. test_search_invalid.py

Asks about the gibberish string `dfghjkgkfjghj`.

- Skill: search or tavily-search.
- Checks: agent invoked search/tavily-search with the gibberish argument; the reply contains a negation phrase (`no results`, `not found`, `gibberish`, `nonsense`, `no meaning`, `unknown`, …).
- Grade: not graded.

13. test_tavily_search.py

Explicitly requests `tavily-search` for "Fetch.ai latest news".

- Skill: tavily-search (must NOT fall back to plain search).
- Checks: agent invoked `(tavily-search …)` with `fetch` in the argument (240 s window); warning (does not fail) if a plain `(search …)` was also called; reply contains Fetch-related keywords AND none of the delivery-error markers (`delivery failed`, `tavily-search failed`, `currently unavailable`, ...) — these would mask a downed external uAgent as a passing test.
- Grade: not graded.

14. test_technical_analysis.py

Requests technical analysis for ticker `AAPL`.

- Skill: technical-analysis.
- Checks: agent invoked `(technical-analysis "AAPL")` with exact ticker match (240 s window); reply mentions the ticker (`aapl` / `apple`) AND at least one TA indicator (`rsi`, `macd`, `sma`, `bullish`, `bearish`, `buy signal`, `trend`, …) AND none of the delivery-error markers.
- Grade: not graded.

Memory

15. test_memory_chromadb.py

Requests the agent to remember a fact tagged with marker `CI-SMOKE-<run_id>`.

- Skill: remember.
- Preparation: harness reads current vector count from `chroma.sqlite3`.
- Checks: agent invoked `(remember …)` with the marker; vector count in the

`embeddings` table grew by ≥ 1.
- Grade: not graded.

16. test_memory_history.py

Sends "Acknowledge with one short line that you received marker `<run_id>`." and verifies the entry in `history.metta`.

- Skill: send (or pin); test exists to verify history bookkeeping.
- Preparation: harness reads current mtime and size of `history.metta`.
- Checks: an s-exp record referencing `REQ-<run_id>` appears in history; agent issued `(send …)` or `(pin …)`; file mtime and size grew.
- Grade: not graded.

17. test_skill_metta.py

Asks the agent to evaluate any short MeTTa expression via the `metta` skill and report the result.

- Skill: metta.
- Checks: agent invoked `(metta …)`; agent then issued a `(send …)` reply describing the result.
- Grade: not graded.

18. test_skill_pin.py

Gives a multi-step task ("restarting servers alpha → beta → gamma, just finished alpha") and expects the agent to track the progress with `pin`.

- Skill: pin.
- Checks: agent invoked `(pin …)` whose argument references either the `run_id` or one of the keywords `step` / `alpha` / `beta` / `gamma` / `restart` / `server` / `done`; agent acknowledged via `(send …)`.
- Grade: not graded.

Working with git

19. test_git_pull_public.py

Agent clones a public repository over anonymous HTTPS, no token.

- Skill: shell.
- Checks: `.git/` directory appears, HEAD points to a real commit, ≥ 1 tracked file in HEAD, origin matches the expected remote URL (normalized — trailing `/` and `.git` ignored).
- **GRADED.** First attempt 120 s, post-clarification 180 s.

20. test_git_local_commit.py

Agent runs `git init`, `git add`, `git commit` locally inside the container.

- Skill: shell.

- Preparation: harness installs git author identity (`OmegaClaw Test <test@omegaclaw.local>`) — without it `git commit` fails.
- Checks: file lands on disk inside the container within 60 s; `.git/` exists and `git log` returns a HEAD line; commit subject contains the `run_id` (warning, not failure); the file is present in the tree; warning (not failure) if no shell call mentioned `git`.
- **GRADED** (commit stage). First attempt 60 s, post-clarification 60 s.

21. test_git_push_to_remote.py

Agent clones a remote, creates a unique branch `qa/run-<id>`, adds a file, commits, and pushes. The harness then verifies via the GitHub REST API that the branch exists with the file, and deletes the branch in teardown.

- Skill: shell.
- Parameters via env vars: `OMEGACLAW_GIT_TOKEN` (token; never appears in code) and `OMEGACLAW_GIT_REMOTE` (default `https://github.com/OmegaSing/Test-Repopo`). Test is skipped if the token variable is unset.
- Checks: file lands on disk inside the container within 120 s; branch present on remote (GitHub API 200); file present on branch; warning (not failure) if the shell call chain did not include `git push`; credentials wiped on teardown.
- **GRADED** (remote-branch stage). First attempt 180 s, post-clarification 180 s — the full chain (clone + branch + write-file + commit + push + GitHub propagation) is heavy under real-LLM latency.

8. Multi-skill tests (combine two or more skills)

These tests exercise sequencing of multiple skills in a single run.

22. test_run_create_dirs.py

Asks the agent to write `mkdirs.sh` and run it. The script must create `test1`, `test2`, `test3` inside `/tmp/test_dirs/`.

- Skills: write-file + shell.
- Preparation: target dir pre-created at `0777` to rule out permission errors.
- Checks: all three directories exist with fresh mtimes; agent invoked `(write-file …)` referencing `mkdirs.sh`; agent invoked `(shell …)` to run the script. Diagnostics print `wf=<count>, sh=<count>, script_present=<bool>, perms=<…>, dirs=<list>` to make stalls obvious — weak models often write only the shebang and never run the script.
- **GRADED.** Both stages 60 s.

23. test_memory_episode.py

Tells the agent that the user's dog Barney lost a baby tooth, waits 60 seconds, then asks to recall when this happened.

- Skills: remember (turn 1) + query or episodes (turn 2).
- Preparation: parse the seed timestamp directly from the leading `("YYYY-MM-DD HH:MM:SS"` of the history.metta block that quotes our REQ-tag (so the reference matches the agent's own clock, not the host's).
- Checks (turn 1): agent invoked `(remember …)` whose argument contains `tooth` or

`Barney` (180 s window — the agent often asks a clarifying question first).
- Checks (turn 2): the reply contains one of the topic words (`dog` / `tooth` / `lost` / `barney`) AND a date in one of: full ISO `YYYY-MM-DD`, short `MM-DD`, hour `HH:`, or English long/abbreviated month-name forms. Independently, either `(query …)` or `(episodes …)` must have been invoked.
- **GRADED.** First attempt 180 s, post-clarification 180 s.

24. test_skill_query.py

Two-turn flow: plant a unique color (`azure-<run_id>`) via `remember`, wait 60 s for embeddings to settle, then ask the agent to recall it via `query` (embedding lookup, not timestamp lookup).

- Skills: remember (turn 1) + query (turn 2).
- Checks (turn 1): agent invoked `(remember …)` carrying the secret color.
- Checks (turn 2): agent invoked `(query …)`; reply mentions the secret color verbatim.
- Grade: not graded.

25. test_skill_episodes.py

Two-turn flow: send a message tagged with `BEACON-<run_id>` (no `remember`), capture the timestamp, wait 90 s, then ask the agent to use `episodes` (timestamp lookup, not `query`) to recall what was discussed at that earlier time.

- Skills: send (turn 1, used as a recordable event) + episodes (turn 2).
- Checks (turn 1): agent issued `(send …)` reply within 60 s — turn is recorded in `history.metta` with a timestamp.
- Checks (turn 2): agent invoked `(episodes …)` for the seed timestamp.
- Grade: not graded.

26. test_complex_weather_flow.py

Four-step pipeline: search NY weather → write `w.txt` with the forecast → write `p.sh` extracting the first Celsius number into `t.txt` → run `p.sh`.

- Skills: search (or tavily-search) + write-file + shell.
- Preparation: `verify_clean` for `/tmp/wflow`; pre-create the directory at `0777`.
- Checks: agent invoked search/tavily-search for NY/weather; `w.txt` exists on disk; history contains a `(write-file …)` referencing `w.txt`; `p.sh` exists with executable bit; history contains `(write-file …)` or `(shell …)` referencing `p.sh`; `t.txt` exists; history contains `(shell …)` running `p.sh`; `t.txt` content is a number in the range `[-60; 120]` (range widened from the original spec because English-language NY sources often report °F); content length ≤ 40 characters.
- **GRADED.** Both stages 60 s.

9. Negative test (live-only)

27. test_memory_no_autoremember.py

Sends a fact-shaped statement (`My favorite color is azure-<run_id>. This is just a casual mention — no need to do anything special.`) and verifies the agent does NOT silently promote it to long-term memory unless it explicitly invokes

`(remember …)`. There is no mock-LLM counterpart — the test is meaningful only against a real model that exercises its own judgement about what to remember.

- Skill: none required (negative).
- Checks: marker lands in `history.metta` within 180 s (confirms HUMAN_MESSAGE was recorded); after a further 60 s settle pause, the test reports one of three outcomes — `agent volunteered remember`, `agent volunteered remember (other content)`, or `no implicit promotion` — based on the presence of an explicit `(remember …)` with the marker and on the ChromaDB vector delta. All three outcomes are passing; the test is informational, surfacing the agent's policy choice rather than enforcing a single behaviour.
- Grade: not graded.

10. Tear down

```
docker rm -f <CONTAINER_NAME>
docker volume rm omegaclaw-memory
```